

ORIGINAL ARTICLE

Impact of $O(n)$ Symmetric Preprocessing on Inversion Reduction and Sorting Algorithm Efficiency: An Experimental Evaluation Across Adaptive and Non-Adaptive Methods

Received on Date Month Year; Accepted on Date Month Year

Abstract

Quadratic sorting algorithms remain relevant in resource-constrained systems but are sensitive to inversion counts. This study investigates the impact of inversions on sorting performance and their reduction through preprocessing. The objective is to assess the effectiveness of this technique for adaptive and non-adaptive algorithms. Eight methods—five classic algorithms plus competitive baselines (`sort_unstable`, `sort`, and $O(n)$ run-detection with reversal)—were compared across ten input topologies (including sawtooth, organ-pipe, few-runs, and real byte data), using vectors from 16 to 10^5 elements for quadratic methods and up to 10^7 for quasilinear methods with $O(n)$ preprocessing, five independent pools per cell, and paired per-vector bootstrap inference. The technique reduced inversions by 40–62% on most non-aligned inputs at about 1.0 ns per element (OrganPipe and FewRuns excepted, at $\approx 0\%$ reduction), with extreme reductions (99.99–100%) restricted to inputs geometrically aligned with the symmetric scan (Reversed, Zigzag). On non-aligned inputs, adaptive quadratic methods gained 14–60% for $n \gtrsim 10^2$ (flat or negative below), with speedups of $\times 386$ – $2,280$ on aligned inputs, while the evaluated $O(n \log n)$ implementations showed mixed results for Merge Sort and systematic degradation for Quicksort. Against `sort_unstable`, preprocessed Insertion Sort remains $\approx 8\times$ slower on random input, and run-detection with reversal matches the method on Reversed, delimiting its practical scope.

Key words: Sorting algorithms, Adaptive sorting, Presorting, Benchmarking, Performance evaluation, Experimental methodology

1. Introduction

Sorting is a fundamental operation in computer science, essential in search systems, databases, and real-time applications [1, 2]. Although methods with $O(n \log n)$ asymptotic complexity, such as Merge Sort and Quicksort, are preferable for large data volumes, quadratic $O(n^2)$ algorithms such as Insertion Sort, Selection Sort, and Bubble Sort remain relevant in specific scenarios [3]. This persistence relates to structural simplicity, low memory consumption through in-place operation, and practical efficiency on small or partially ordered datasets [3, 4].

The practical efficiency of a sorting algorithm does not depend exclusively on its asymptotic complexity; it is also influenced by properties of the input sequence [5]. In this context, an inversion, defined as a pair satisfying $A[i] > A[j]$ for $i < j$, constitutes a quantitative measure of the disorder degree of an array [1, 6]. The inversion count characterizes different levels of presortedness, since sequences with more inversions exhibit greater disorder relative to the final order [6]. This characteristic can significantly influence execution cost, especially for algorithms whose operation count depends on input disorder [5].

Despite this relation between input disorder and execution cost, quadratic algorithms may exhibit limited adaptability to initial data conditions [5, 6]. In particular, inputs with many inversions can increase required movements during sorting, degrading performance of methods such as Insertion Sort [3]. In this scenario, a preliminary step capable of reducing input disorder may decrease the subsequent work of the sorting algorithm. The research question is therefore: to what extent does a preprocessing step aimed at inversion reduction impact the practical performance of quadratic and quasilinear sorting algorithms?

Formally, let C_{original} denote the temporal cost of directly sorting the input. By introducing symmetric preprocessing with cost C_{pre} , the subsequent sorting algorithm operates on a modified vector, spending a reduced time C_{sort} . The objective of this work is to propose, implement, and evaluate a low-cost $O(n)$ symmetric preprocessing algorithm designed to reduce the inversion count of an input vector. We empirically verify whether the hybrid approach is more time-efficient than directly applying the same conventional algorithm, identifying the practical conditions under which $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$ holds. The comparison is extended to the fastest available general sorters

(`sort_unstable/sort`) and to $O(n)$ run-detection with reversal; a formal closed-form theory of expected reduction is left as future work.

The relevance of this study lies in investigating a strategy capable of improving practical sorting behavior without modifying fundamental structure or introducing high additional cost. Reducing disorder before the main sort may decrease operations on unfavorable inputs, especially in methods sensitive to inversion counts [5, 7]. The study also contributes to adaptive-algorithm analysis by investigating conditions under which preprocessing cost is compensated by reduced sorting work [6, 7].

The research adopts a quantitative, applied, and experimental approach [8, 9]. The study comprises formal modeling of the preprocessing algorithm and its implementation in Rust. Experimental evaluation used automated benchmarks with the Criterion library, employing synthetic vectors ranging from 16 to 10,000,000 elements with controlled inversion levels, covering worst cases, partial ordering, small inputs where quadratic methods are actually used, and adversarial structured topologies inspired by classical sorting-test patterns, plus a real byte-data asset. Experiments compared execution time of the hybrid approach against directly applied sorting algorithms on the same inputs, and against the fastest available general sorters.

This article is structured as follows. Section 2 presents theoretical background. Section 3 describes methodology, formalization of the proposed preprocessing algorithm, evaluated sorting algorithms, and experimental procedures. Section 4 presents and discusses results. Finally, Section 5 presents conclusions.

2. Background and Related Work

This section presents the theoretical foundations: inversions as a disorder measure, adaptive-sorting principles, characteristics of evaluated algorithms including quadratic and quasilinear methods, preprocessing concepts and symmetric-preprocessing principles, and related work situating this contribution.

2.1. Inversions and Adaptive Sorting

Practical efficiency of traditional sorting algorithms often depends on initial element disposition [3, 5]. To formalize the disorder degree, the classical concept of *inversion* is used [1]. Given a vector a of n elements, an inversion is formally a pair (i, j) such that $i < j$ and $a[i] > a[j]$ [1, 2]. The total inversion count I measures input disorder and relates directly to effort required to sort via local operations [6].

The relation between inversion volume I and execution complexity gives rise to *adaptivity* [5]. An algorithm is adaptive when its time complexity improves markedly in the presence of partial presortedness [6]. A classical example is *Insertion Sort*, consuming $O(n + I)$ operations. On a strictly ordered vector ($I = 0$) it reaches linear $O(n)$ time, while in the worst case ($I = O(n^2)$) it degrades to quadratic time [1, 3].

2.2. Evaluated Algorithms

To evaluate symmetric-preprocessing impact across sorting paradigms, a representative set with distinct time complexity, adaptivity, and sensitivity to initial order was selected, covering both methods whose efficiency may be directly influenced by disorder and methods less dependent on it.

In the quadratic $O(n^2)$ worst-case group, *Bubble Sort*, *Insertion Sort*, and *Selection Sort* were evaluated [3]. *Insertion Sort* is adaptive in the inversion count ($O(n + I)$); *Bubble Sort* with early exit is adaptive in the number of passes—which is governed by the maximum left-displacement of any element rather than by the total I —while *Selection Sort* is non-adaptive and maintains fixed comparisons regardless of initial configuration [5]. This distinction matters for interpreting results: inversion elimination translates directly into fewer shifts for Insertion Sort, whereas Bubble Sort gains arise only when preprocessing collapses the maximum displacement (as in Zigzag and Reversed inputs).

In the quasilinear group, *Merge Sort* with $O(n \log n)$ time and *Quicksort* with $O(n \log n)$ average performance (though $O(n^2)$ worst case) were evaluated [3]. Their inclusion verifies whether effects observed in quadratic algorithms remain relevant at lower asymptotic cost, extending analysis beyond inversion-count-related algorithms.

Thus the set covers three relevant dimensions: adaptive quadratic algorithms (*Bubble Sort* and *Insertion Sort*), a non-adaptive quadratic algorithm (*Selection Sort*), and quasilinear algorithms based on distinct strategies (*Merge Sort* and *Quicksort*). This composition verifies not only whether symmetric preprocessing reduces inversions, but to what extent this reduction converts into gains for algorithms with different dependence on initial order.

2.3. Symmetric Preprocessing Concept

Preprocessing, in algorithm design and analysis, refers to a preliminary transformation of input data before the main algorithm [3]. Its purpose is to alter instance disposition, making subsequent resolution simpler or more efficient. Conceptually, preprocessing acts analogously to prior organization of a work environment: although preparation requires initial cost, reduced main-processing effort compensates it.

For viability in sorting, preprocessing temporal cost must be strictly dominated by the succeeding sorting complexity [2, 3]. An ideal preprocessing should exhibit linear $O(n)$ time, ensuring overhead is asymptotically negligible relative to total sorting cost—whether $O(n \log n)$ average quasilinear or $O(n^2)$ worst-case quadratic [1].

In this research, preprocessing is termed *symmetric* because it applies a bidirectional mirrored scan from vector ends. This restructuring aims to reduce total inversions I by moving discrepant elements closer to final destinations in a single $O(n)$ pass. By providing a partially reordered structure, it seeks to mitigate comparisons and swaps in both adaptive and non-adaptive methods.

2.4. Related Work

Literature presents different approaches to characterize sequence sortedness and exploit it. Mannila [6] studied presortedness and established measures quantifying disorder forms. The author defined general properties for presortedness measures and introduced optimal sorting relative to a measure. Among measures is inversion count $Inv(X)$. The work presented optimal algorithms for different measures and showed *Insertion Sort* optimal for three natural measures, establishing foundations for adaptive algorithms [6].

Estivill-Castro and Wood [5] surveyed adaptive sorting, considering input disorder degree. The work presented main concepts, disorder measures, and established results on *adaptive sorting*, synthesizing theoretical and practical results including

relations between disorder measures and algorithms exploiting presortedness. The survey records that several algorithms exhibit complexity dependent on both input size and disorder, and that certain adaptive algorithms perform competitively on random sequences with significant gains on nearly sorted inputs.

In an approach directly related to preliminary transformations, Hwang, Yang, and Yeh [7] investigated *presorting* algorithms from an average-case perspective, using inversion-count reduction as performance measure. The authors formulated quantification of preliminary-step effects via average inversion reduction, analyzing steps of known algorithms including *Shellsort* and *Quicksort*, and derived expectation, variance, and generating functions of that reduction. Their framework could in principle host a closed-form derivation of the expected reduction of symmetric preprocessing under uniform random input; Section 3.4 derives the mirrored-swap first-order term ($\approx n^2/12$), while the full generating-function treatment remains future work. Empirically, the average reduction on random input was $\sim 45\%$ of the initial $\Theta(n^2)$ inversions (Table 3)—a large absolute $\Theta(n^2)$ reduction produced by a linear pass, whose precise expectation requires the generating-function machinery of [7].

Recent literature reinforces practical relevance of microarchitectural effects. Edelkamp and Weiss [10] showed branch mispredictions can dominate Quicksort cost and proposed *BlockQuicksort*, providing basis for degradation hypotheses for *Selection Sort* and Quicksort in this work. Auger et al. [11] analyzed *Timsort* worst-case complexity, showing exploitation of pre-existing runs remains an active sorting-engineering theme. LaMarca and Ladner [12] showed cache influence on sorting performance, also discussed by Kaligosi and Sanders [13] when quantifying branch-misprediction impact. These works position the present low-cost technique as a preprocessing-heuristic instance whose trade-off must be jointly evaluated at logical (inversions) and microarchitectural levels.

Differing from mapped approaches, this research evaluates a Symmetric Preprocessing algorithm acting on direct inversion mitigation in linear time, as one low-cost preprocessing instance. Run-detection is included as a competitive baseline (Section 3.2, Table 5); systematic comparison against Shell-type passes and a formal gap analysis remain future work.

3. Experimental Methodology

This work follows an applied, quantitative, experimental evaluation design [8, 9]: it controllably manipulates input topology and size (independent variables) and observes execution time and inversion counts (dependent variables) under a fixed seed, paired vector generation, and a standardized environment. From an evaluation-methodology standpoint, the study is framed as a cost–benefit assessment of a preprocessing pass: the measured total $C_{\text{pre}} + C_{\text{sort}}$ internalizes preprocessing overhead, the paired design controls input variability as a confounder, and correctness is checked outside the timed region. Three working hypotheses guide the analysis: (H1) adaptive quadratic algorithms benefit where absolute inversion reduction ΔI is large; (H2) the non-adaptive quadratic algorithm is neutral to inversion reduction; (H3) for quasilinear methods the overhead C_{pre} dominates unless structural disorder is extreme.

Table 1. Experimental environment and tools.

Component	Configuration / Specification
Operating system	Windows 11 Pro (build 10.0.26200)
Processor	AMD Ryzen 7 8700G
Memory	16 GB DDR5 5600MT/s
Language	Rust 1.96.0 (release profile)
Measurement tools	Criterion 0.5 and Project CLI Tool
Data generation	Seeds 42–46 (ChaCha8Rng), paired inputs

Table 2. Input-topology generators (n = size, 0-based index i).

Topology	Exact Generator
Random	shuffle of $[0, n-1]$ with ChaCha8Rng
Turtles	$a[i] = i + n$ if $i < n/2$, else $a[i] = i \bmod 10$
Zigzag	$a[i] = i$ if i even, else $a[i] = n - i$
Almost Sorted	$[0, n-1]$ with $n/100$ random pair swaps
Duplicates	$a[i] \sim \mathcal{U}\{0, 1, 2\}$ i.i.d.
Reversed	$a[i] = n - i$ (strictly decreasing)
Sawtooth	$a[i] = i \bmod 16$
OrganPipe	$a[i] = i$ if $i < n/2$, else $n - 1 - i$
FewRuns	4 reversed blocks at quartiles
Real	frozen byte-asset prefix

3.1. Experimental Configuration

Experiments ran in a controlled environment whose hardware, OS, and tool specifications appear in Table 1. To reduce variability, power plan was kept at high performance and background processes minimized. Code was compiled in *release* mode with maximum optimization (`opt-level = 3`). Input generation used a fixed seed, ensuring reproducibility and correspondence across experimental conditions.

To evaluate resilience under different entropy degrees, ten structural input patterns were implemented with the exact generators in Table 2 (0-based indices, values in `i32`). For each topology-size combination, a single pool of 50 vectors is generated with **ChaCha8Rng** seeded by $42 \oplus n \oplus \text{type}$ and shared by all algorithms and both variants (pure and preprocessed), ensuring paired comparability. Deterministic generators (Reversed, Zigzag, Turtles) yield identical vectors across the 50 pool slots; stochastic generators (Random, Duplicates, Almost Sorted) advance the same RNG stream, so slot k differs across slots but is identical across algorithms and variants for fixed k . The core matrix is replicated with five independent pools (seeds 42–46) for between-pool stability analysis (Section 3.3).

Quasilinear algorithms (*Merge Sort* and *Quicksort*) and the standard-library baselines were measured at $n \in \{16, \dots, 10,000,000\}$. For quadratic algorithms (*Insertion Sort*, *Bubble Sort*, *Selection Sort*, *DescReverse*), measurements were restricted to $n \leq 100,000$: at $n = 10^6$ each execution would take minutes, precluding systematic collection of statistically significant samples. This restriction is documented and does not compromise comparison, as gain orders of magnitude already stabilize at $n = 100,000$. The $n=10^7$ points exercise the out-of-cache regime (≈ 40 MB per `i32` vector); the Real topology is capped at its native asset size ($N=477,104$, no tiling).

Isolated preprocessing cost (C_{pre}) was additionally characterized at $n \in \{1,000, 5,000, 10,000, 20,000, 100,000, 1,000,000\}$ (Table 7 and Fig. 2) to evidence linearity; the $n=20,000$ point was collected exclusively for this cost curve via CLI tool (`seed 42`), under the same paired-generation protocol, and is not part of the main sorting-time comparison matrix.

3.2. Evaluated Algorithm Specifications

To ensure internal validity, timing stability, and reproducibility, all sorting algorithms were implemented in Rust (version 1.96.0), operating directly on mutable slices of 32-bit integers (`&mut [i32]`). Implementations avoided high-level external abstractions, standardizing memory handling and adopting idiomatic instruction-level optimizations.

3.2.1. Quadratic Algorithms

- **Bubble Sort:** Optimized adaptive version. Successive scans reduce upper bound each iteration ($n - 1 - i$). Adaptivity is ensured by a boolean flag (`swapped`) monitoring swaps per pass; if a scan completes with no element permutation, execution halts early, ensuring $O(n)$ on presorted vectors.
- **Insertion Sort:** Traditional direct-insertion by shifting. For each element of the unordered sub-slice, current value is stored in a temporary variable and strictly greater antecedents are shifted right. Search and shifting end once correct position is found or left bound ($j = 0$) is reached, minimizing write overhead versus successive-swap implementations.
- **Selection Sort:** Classical non-adaptive in-place form. At each outer iteration i , the algorithm scans unordered segment $[i + 1, n - 1]$ to identify minimum-element index. Swap with position i executes conditionally only if minimum resides at a distinct index. Comparison cost remains strict $O(n^2)$, regardless of presortedness.

3.2.2. Quasilinear Algorithms

- **Merge Sort:** Top-down divide-and-conquer with stable ordering guarantee. To eliminate continuous dynamic-allocation overhead distorting timings, implementation allocates a single temporary auxiliary vector of size n at function entry, passed by mutable reference through recursive calls. Sub-slice midpoints use right bit-shifting ($(\text{end} - \text{start}) \gg 1$), avoiding integer division. Merge copies current-slice data to auxiliary buffer and rebuilds ordered segment with three index pointers.
- **Quicksort:** Recursive implementation using three-way partitioning (*Dutch National Flag* / Dijkstra’s algorithm) with median-of-three pivot selection. Pivot is median among initial, central, and final subvector elements. Three-way partitioning rearranges into three contiguous regions: less than, equal to, and greater than pivot, maintaining `lt`, `i`, `gt` indices. This yields $O(n \log n)$ average time and robustness to many duplicate keys (no worst-case guarantee: median-of-three does not preclude $O(n^2)$ on adversarial inputs). Tail-call optimization iterates on the larger partition and recurses on the smaller.

3.2.3. Competitive Baselines

- **sort_unstable:** the Rust standard-library unstable sort (pdqsort-based pattern-defeating quicksort), the fastest available general sorter and the reference any proposal must beat.
- **sort:** the stable standard-library sort (driftsort-based), included as a worst-case-stable reference.
- **desc_reverse:** $O(n)$ strictly-descending run check; if the input is strictly decreasing it is reversed in place, then Insertion Sort runs (otherwise Insertion Sort runs directly). This tests whether trivial run-detection explains the gains on Reversed.

In all implementations, preliminary boundary checks (*early returns*) ensure slices with $n < 2$ return immediately without function calls or unnecessary allocations.

3.3. Experimental Protocol and Inversion Counting

Empirical measurement used the *Criterion.rs* framework: warm-up cycles mitigate cache effects, and bootstrap resampling estimates 95% confidence intervals (`confidence_level=0.95`, `significance_level=0.05`). Each Criterion sample clones the next pool vector in rotation (`iter_batched`, per-iteration clone outside the timed region), so the 50-vector pool feeds all samples while cloning cost is never timed; only $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$ is timed in the preprocessed variant.

Measurement configuration was profile-adjusted (configured values; Criterion may extend effective measurement to reach statistical stability): 50 samples with 2s warm-up and 5s measurement window for most scenarios; for quadratic algorithms at $n = 100,000$ (iterations up to ≈ 2 s for Selection Sort), 30 samples with 1s warm-up and 5s window; and for quasilinear methods at $n \geq 10^6$, 50 samples with 2s warm-up and 10s window. Mean, median, and 95% CIs are computed per cell; tables report mean $\pm$ CI half-width.

Statistical criterion: for each cell and each pool, pure vs. preprocessed variants are compared on the *same* vectors (paired design, $R=50$ slots). The effect statistic is the mean per-vector log-ratio $\log(T_{\text{pre}}/T_{\text{pure}})$, with a 95% percentile-bootstrap confidence interval ($B=10,000$ resamples, deterministic per-cell RNG); arm order is alternated across repetitions to control drift. An effect is reported as significant when the paired CI excludes zero, marked * in Tables 4–9; † marks effects significant *and* sign-stable across all five pools. Cells significant under pairing but unstable across pools (e.g. Quicksort/AlmostSorted at $n=10^6$) are explicitly flagged as non-robust. This replaces the earlier conservative non-overlap rule, which the paired analysis supersedes (notably, Selection/Zigzag at $n=10,000$ is non-significant under pairing).

Beyond timing, correctness of each scenario was verified outside the timed region: for every (algorithm, topology, size), the sorted output is checked element-wise ($a[0] \leq a[1] \leq \dots$) and against `sort_unstable` in both pure and preprocessed variants. Inversion counts of each shared-pool vector, before and after preprocessing, are recorded in harness companion files, ensuring inversion tables derive from exactly the same timed vectors.

Exact inversion-count quantification uses a modified-MergeSort-based $O(n \log n)$ implementation [3]. During merging of two ordered subvectors L and R , when $R[j]$ is less than $L[i]$, exactly $|L| - i$ is added to the counter. This eliminates $O(n^2)$ direct-counting bottleneck, enabling evaluation of 10^5 – 10^6 -element vectors.

3.4. Symmetric Preprocessing Formalization

The proposed symmetric preprocessing operates as a deterministic single-pass scan over input vector A of size n . Its goal is restructuring via mirrored swaps and local adjustments before evaluated sorting algorithms. Procedural structure is formalized in Algorithm 1.

The algorithm iterates over the first half, scanning $i \in [0, \lfloor n/2 \rfloor - 1]$. Each iteration computes the symmetric index at the opposite end, $j = (n - 1) - i$. Loop execution comprises three structured conditionals: (1) **Extreme symmetric swap:** if $A[i] > A[j]$, direct swap places smaller element in left segment and larger in right; (2) **Left adjacent adjustment:**

Algorithm 1 Symmetric Preprocessing Algorithm

Require: Vector A of size n
Ensure: Partially reorganized vector A

```

1:  $n \leftarrow \text{size of } A$ 
2: if  $n < 2$  then
3:   return
4: end if
5:  $\text{last} \leftarrow n - 1$ 
6:  $\text{mid} \leftarrow \lfloor n/2 \rfloor$ 
7: for  $i \leftarrow 0$  to  $\text{mid} - 1$  do
8:    $j \leftarrow \text{last} - i$ 
9:   if  $A[i] > A[j]$  then
10:     $\text{swap}(A[i], A[j])$ 
11:   end if
12:   if  $i + 1 < \text{mid}$  then
13:     if  $A[i] > A[i + 1]$  then
14:        $\text{swap}(A[i], A[i + 1])$ 
15:     end if
16:   end if
17:   if  $j - 1 > \text{mid}$  then
18:     if  $A[j] < A[j - 1]$  then
19:        $\text{swap}(A[j], A[j - 1])$ 
20:     end if
21:   end if
22: end for

```

for $(i + 1) < \lfloor n/2 \rfloor$, if $A[i] > A[i + 1]$, swap adjacent elements; (3) **Right adjacent adjustment:** for $(j - 1) > \lfloor n/2 \rfloor$, if $A[j] < A[j - 1]$, swap adjacent elements.

Boundary conditions ensure adjacent swaps operate strictly within respective subdomains without crossing the midpoint: the left adjustment covers pairs up to $(\text{mid}-2, \text{mid}-1)$ and the right adjustment from $(\text{mid}+1, \text{mid}+2)$ onward, so neither side touches a midpoint-crossing adjacent pair and the procedure is left-right symmetric. Computationally, the algorithm executes exactly $\lfloor n/2 \rfloor$ iterations. Each step performs at most 3 comparisons and 3 swaps, yielding $T(n) = O(n)$ worst-case time. Since restructuring is in-place, auxiliary space is strictly $O(1)$. A useful monotonicity property is that swapping an inverted mirrored pair never increases I : it removes the (i, j) inversion and each intermediate k with $a_j < a_k < a_i$ contributes a net reduction, so the pass is a descent step on I .

For uniform random input the mirrored swaps admit a first-order expectation. Condition on a mirrored swap at (i, j) with $m = j - i - 1$ interior positions (the swap itself occurs with probability $1/2$): the swap removes the (i, j) inversion, and each interior k contributes expected net reduction $2/3$ (each of the two affected pairs (i, k) , (k, j) improves with probability $1/3$ given the swap). Taking expectations over the $n/2$ mirrored pairs gives $\mathbb{E}[\Delta I] = n^2/12 + O(n)$, about one third of the $\mathbb{E}[I] = n^2/4$ baseline. The measured 45.19% on Random exceeds this first-order term because the adjacent adjustments contribute further reduction; a closed form including them is future work (Section 5).

For adaptive methods with cost $\approx c$ per inversion ($c \approx 0.21$ ns measured, Section 4.2), preprocessing pays off iff $C_{\text{pre}} < c\Delta I$. With $C_{\text{pre}} \approx \alpha n$ ($\alpha \approx 1$ ns/element overall, 0.5–3.0 ns by topology, Table 8), the rule of thumb is $\Delta I \gtrsim 5n$ inversions removed; topology-specific breakevens follow from Table 8 (e.g. 144,000 for Random, 28,000 for OrganPipe at $n=10,000$). Every $n=10,000$ outcome in Tables 4–6 obeys this rule: Random ($\Delta I=11.3\text{M}$), Sawtooth (12.5M), and Duplicates (10.3M)

gain, while OrganPipe (10,000) and FewRuns (5,000) fall below breakeven and are neutral.

3.5. Threats to Validity

Internal: single machine, OS (Windows 11), compiler (Rust 1.96.0, release, `opt-level=3`) and i32 data; CPU frequency scaling, SMT affinity, and Windows timer resolution were not controlled beyond high-performance power plan and background minimization. *Construct:* C_{pre} is measured per topology within the same Criterion harness (Table 8), so per-cell C_{pre} vs. ΔC_{sort} decomposition holds for the reported matrix; only i32 keys and one implementation per algorithm were tested. *External:* nine synthetic topologies plus one real byte asset, five independent pools per cell, quadratic methods capped at $n \leq 10^5$, Duplicates fixed at $k=3$ values; $n=10^6$ vectors (≈ 4 MB) are cache-resident on the 8700G while $n=10^7$ exercises the out-of-cache regime (Fig. 4). *Statistical:* paired bootstrap inference with between-pool stability for the published tables ($n=10^4$, $n=10^6$); remaining sizes rely on Criterion CIs. *Measurement:* no hardware counters (`perf_event_open` is Linux-specific); branch-predictor and cache attributions are hypotheses, not measurements. All experiments are reproducible via `cargo bench` and routines in the project repository [14].

4. Results and Discussion

This section analyzes experimental results, evaluating symmetric-preprocessing impact on data structure and algorithm execution time. Discussion follows an analytical structure from direct inversion-reduction impact and geometric-bias interpretation in the reversed scenario, through total sorting-time measurement in quadratic and quasilinear algorithms, to microarchitectural effects such as locality and CPU caches. The goal is consolidating scenarios with real gains and understanding physical and algorithmic limitations.

4.1. Inversion Reduction and Structural Bias

Fig. 1 illustrates inversion variation on 10,000-element vectors, with exact quantitative values and reduction percentages in Table 3. On non-aligned inputs with unstructured disorder (Random, Turtles, Almost Sorted, Duplicates, Real, Sawtooth) preprocessing removes 40–62% of inversions—a large absolute $\Theta(n^2)$ reduction. Two non-aligned topologies resist the scan almost entirely: *OrganPipe* (0.04%) and *FewRuns* (0.16%), whose disorder geometry the mirrored comparisons never engage. The near-total eliminations in *Reversed* (100%) and *Zigzag* (99.99%) occur precisely where disorder geometry coincides with the mirrored-scan geometry, and must be read as best-case alignment rather than typical behavior; the primary generalizable evidence is the Random, Almost Sorted, Real, and Sawtooth columns.

Since Algorithm 1 directly swaps $a[i]$ and $a[n - 1 - i]$, a strictly reversed vector (where $a[i] > a[n - 1 - i]$ for all $i < n/2$) undergoes complete geometric reversal in $O(n)$ operations. Resolving Reversed stems from preprocessing-transformation geometry, an ideal case where disorder topology perfectly matches applied symmetry.

Behavior in *Zigzag* can also be interpreted via this structural property. Although its topology is not a global reversal like *Reversed*, systematic alternation between differently oriented blocks produces disorder relations partially captured by symmetric-position comparisons. Direct swapping between $a[i]$ and $a[n - 1 - i]$ thus simultaneously corrects many out-of-order

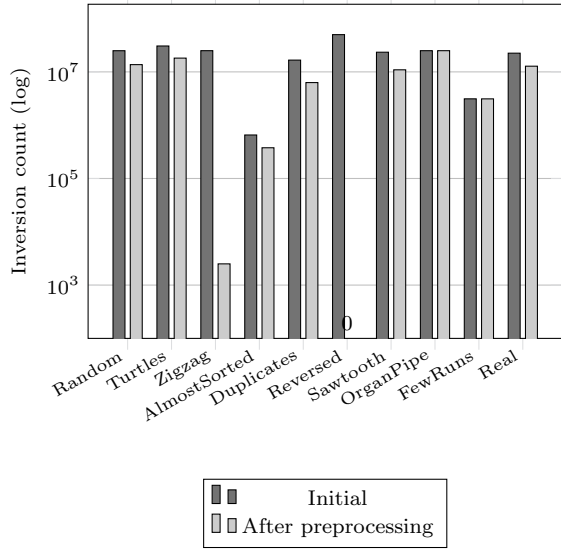


Figure 1. Inversion counts before and after symmetric preprocessing for 10,000-element vectors (log scale; the Reversed post value is exactly 0—annotated above the axis floor—and has no bar on the log scale).

Table 3. Inversion reduction by symmetric preprocessing on 10,000-element vectors (mean of 50 paired-pool vectors; same seed as Table 4).

Type	Inversions	Post-pre	Reduction
Random	24.960.120	13.681.366	45,19%
Turtles	30.613.750	18.133.750	40,77%
Zigzag	24.997.500	2.501	99,99%*
Almost Sorted	654.922	377.149	42,41%
Duplicates	16.633.993	6.300.346	62,12%
Reversed	49.995.000	0	100,00%**
Sawtooth	23.400.000	10.937.472	53,26%
OrganPipe	24.995.000	24.985.004	0,04%
FewRuns	3.122.500	3.117.505	0,16%
Real	22.447.037	12.789.942	43,02%

* 99.99% (2,501 remaining); ** 100.00% exact (zero remaining). Rounding difference precludes equivalence interpretation. OrganPipe (0.04%) and FewRuns (0.16%) are structurally invisible to the mirrored scan (Section 4.1).

relations, reducing inversions from 24,997,500 to only 2,501, approximately 100% (99.99%) reduction.

The result shows preprocessing efficacy is not restricted to complete reversal: it also manifests in topologies strongly aligned with transformation symmetry, even partially. This helps explain particularly high gains later observed in Insertion Sort and Bubble Sort for this input.

4.2. Execution-Time Impact and Preprocessing Cost

Gain is defined as $(T_{\text{pure}} - T_{\text{pre}})/T_{\text{pure}} \times 100\%$; equivalent speedups $T_{\text{pure}}/T_{\text{pre}}$ are given for saturated cells. Table 4 presents mean execution time and gain per algorithm on 10,000-element vectors, showing how preprocessing efficiency is conditioned by native complexity and adaptivity. On non-aligned inputs, *Insertion Sort* gained 28.6–60.3% (Random +43.9%) and *Bubble Sort* 14.9–26.2%. The largest benefits occur on aligned inputs: in *Zigzag*, preprocessing reduced *Insertion Sort* from 5,249.97 μs to 13.60 μs (+99.7%, $\times 386$ speedup) and

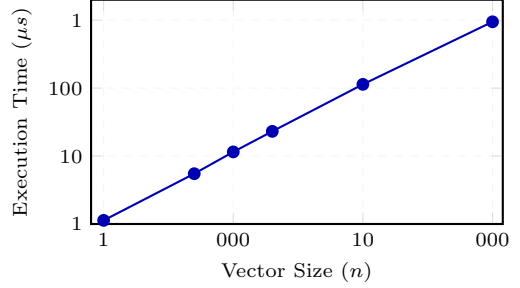


Figure 2. Mean preprocessing execution time vs. vector size (n), log-log scale evidencing $O(n)$ linear behavior. Means over ten input types, CLI tool seed 42.

Bubble Sort from 18,469.42 μs to 16.00 μs (+99.9%, $\times 1,155$ speedup); in *Reversed*, $\times 837$ and $\times 2,280$ speedups respectively (reported as +99.9% and +100.0% after rounding). In contrast, $O(n \log n)$ methods showed mixed results for *Merge Sort* (degradation up to 14.2% and gains up to 15.7% at $n=10,000$; gains up to 18.6% at $n=10^6$, Table 9). Since divide-and-conquer already ensures high native efficiency, preliminary-step cost tends to outweigh benefits in low-structure distributions. Significance follows the paired bootstrap criterion (Section 3.3): non-significant cells at $n=10,000$ are Merge/Duplicates (−1.2%) and Selection in Random, Almost Sorted, Duplicates, and Zigzag; Selection/Turtles (+3.1%) is significant in seed 42 but unstable across pools, and Quicksort/AlmostSorted (−4.5%) is non-significant under pairing despite its point estimate.

Preprocessed Insertion Sort on Random input (2,921.09 μs) remains $\approx 8.1\times$ slower than pure Quicksort (360.74 μs), and *sort_unstable* itself (80.41 μs) is $\approx 36\times$ faster than the best hybrid on the same input—so no same-algorithm gain translates into competitiveness against the standard sorter. On Reversed, however, *desc.reverse* (11.03 μs pure) matches the preprocessed Insertion Sort (12.81 μs): trivial run-detection captures the entire aligned-input gain without the method, confirming that the extreme speedups are a property of the input geometry, not of the preprocessing.

Sawtooth behaves like the unstructured topologies (Insertion +52.3%, Bubble +33.8%, *desc.reverse* +51.7%), Real tracks Random closely (Insertion +42.2%, Bubble +13.6%), while OrganPipe and FewRuns are neutral for every method (−5.2% to +2.5%, mostly non-significant)—consistent with their $\approx 0\%$ inversion reduction. The standard sorters degrade under preprocessing on the new topologies (down to −71.3% for *sort* on OrganPipe), since the $O(n)$ overhead buys them nothing.

Practical viability rests on low, highly predictable symmetric-step cost. As shown in Table 7 and Fig. 2, preprocessing execution time is strictly linear $O(n)$ in vector size, with unit time between 0.95 and 1.15 ns per element, lowest at 10^6 elements (≈ 0.95 ns), indicating no memory-locality bottlenecks at scale. The scan thus imposes negligible overhead—only 11.51 μs for 10,000 elements—amply compensated by millisecond savings in adaptive sorting.

The heterogeneity is itself informative: the scan’s swap conditions are data-dependent branches, so Random and Duplicates inputs stall the pipeline (≈ 30 and 28 μs at $n=10,000$) while structured inputs cost ≈ 5 –7 μs . Consequently, per-cell overhead must be read from Table 8, not from the Table 7 mean: e.g. Merge/Random at $n=10,000$ slows by +11.46 μs

Table 4. Mean time per execution (in μs , mean $\pm$ 95% CI half-width) and preprocessing gain for 10,000-element vectors. Source: Criterion, seed 42. Gain marked * when the paired bootstrap 95% CI excludes zero (seed 42) and † when additionally sign-stable across all five pools.

Algorithm	Type	Pure	With pre	Gain
Merge Sort	Random	451.57 $\pm$ 1.66	463.03 $\pm$ 1.49	-2.5% †
Merge Sort	Turtles	122.36 $\pm$ 0.60	139.68 $\pm$ 1.82	-14.2% †
Merge Sort	Zigzag	135.37 $\pm$ 0.66	114.16 $\pm$ 0.51	+15.7% †
Merge Sort	Almost Sorted	137.14 $\pm$ 1.58	142.22 $\pm$ 0.93	-3.7% †
Merge Sort	Duplicates	201.45 $\pm$ 0.73	203.87 $\pm$ 1.66	-1.2%
Merge Sort	Reversed	126.92 $\pm$ 0.96	122.77 $\pm$ 1.09	+3.3% †
Quicksort	Random	360.74 $\pm$ 1.18	378.06 $\pm$ 1.34	-4.8%*
Quicksort	Turtles	140.18 $\pm$ 1.75	151.37 $\pm$ 1.47	-8.0%*
Quicksort	Zigzag	381.21 $\pm$ 4.46	379.51 $\pm$ 4.31	+0.5%*
Quicksort	Almost Sorted	217.15 $\pm$ 1.39	226.97 $\pm$ 1.35	-4.5%
Quicksort	Duplicates	41.45 $\pm$ 0.35	55.70 $\pm$ 0.56	-34.4% †
Quicksort	Reversed	328.39 $\pm$ 4.70	454.47 $\pm$ 9.17	-38.4% †
Insertion Sort	Random	5209.69 $\pm$ 40.87	2921.09 $\pm$ 18.70	+43.9% †
Insertion Sort	Turtles	6330.42 $\pm$ 40.35	3801.57 $\pm$ 15.96	+40.0% †
Insertion Sort	Zigzag	5249.97 $\pm$ 31.16	13.60 $\pm$ 0.16	+99.7% †
Insertion Sort	Almost Sorted	188.55 $\pm$ 4.27	134.69 $\pm$ 3.60	+28.6% †
Insertion Sort	Duplicates	3673.33 $\pm$ 32.52	1456.81 $\pm$ 19.69	+60.3% †
Insertion Sort	Reversed	10725.42 $\pm$ 98.39	12.81 $\pm$ 0.23	+99.9% †
Bubble Sort	Random	23314.17 $\pm$ 91.56	19847.68 $\pm$ 62.53	+14.9% †
Bubble Sort	Turtles	18790.39 $\pm$ 62.61	13857.28 $\pm$ 72.47	+26.2% †
Bubble Sort	Zigzag	18469.42 $\pm$ 67.66	16.00 $\pm$ 0.18	+99.9% †
Bubble Sort	Almost Sorted	16513.40 $\pm$ 137.86	13788.51 $\pm$ 51.03	+16.5% †
Bubble Sort	Duplicates	20765.80 $\pm$ 84.42	16600.83 $\pm$ 97.35	+20.1% †
Bubble Sort	Reversed	21031.38 $\pm$ 140.17	9.22 $\pm$ 0.19	+100.0% †
Selection Sort	Random	21106.62 $\pm$ 176.37	21226.87 $\pm$ 113.10	-0.6%
Selection Sort	Turtles	20797.92 $\pm$ 86.62	20156.05 $\pm$ 106.47	+3.1%*
Selection Sort	Zigzag	20470.58 $\pm$ 52.68	20447.90 $\pm$ 62.51	+0.1%
Selection Sort	Almost Sorted	20617.08 $\pm$ 87.96	20704.93 $\pm$ 119.86	-0.4%
Selection Sort	Duplicates	21299.72 $\pm$ 150.99	21516.23 $\pm$ 169.19	-1.0%
Selection Sort	Reversed	19150.38 $\pm$ 353.49	20900.45 $\pm$ 113.30	-9.1% †

Table 5. Competitive baselines: mean time per execution (in μs , mean $\pm$ 95% CI half-width) and preprocessing gain for 10,000-element vectors. `sort_unstable` and `sort` are the Rust standard-library sorters; `desc_reverse` is Insertion Sort preceded by $O(n)$ descending-run detection with reversal. Same significance marks as Table 4.

Algorithm	Type	Pure	With pre	Gain
<code>sort_unstable</code>	Random	80.41 $\pm$ 2.35	107.55 $\pm$ 0.48	-33.8% †
<code>sort_unstable</code>	Turtles	47.26 $\pm$ 0.28	51.34 $\pm$ 0.28	-8.7% †
<code>sort_unstable</code>	Zigzag	70.63 $\pm$ 0.46	66.09 $\pm$ 0.29	+6.4% †
<code>sort_unstable</code>	Almost Sorted	76.10 $\pm$ 0.36	83.16 $\pm$ 0.43	-9.3% †
<code>sort_unstable</code>	Duplicates	15.08 $\pm$ 0.28	40.83 $\pm$ 0.19	-170.8% †
<code>sort_unstable</code>	Reversed	3.75 $\pm$ 0.12	7.86 $\pm$ 0.19	-109.7% †
<code>sort</code>	Random	112.00 $\pm$ 0.69	140.99 $\pm$ 0.82	-25.9% †
<code>sort</code>	Turtles	23.44 $\pm$ 0.25	27.79 $\pm$ 0.17	-18.5% †
<code>sort</code>	Zigzag	93.45 $\pm$ 0.70	47.60 $\pm$ 0.29	+49.1% †
<code>sort</code>	Almost Sorted	124.33 $\pm$ 0.64	126.66 $\pm$ 0.47	-1.9%
<code>sort</code>	Duplicates	17.85 $\pm$ 0.55	43.79 $\pm$ 0.28	-145.3% †
<code>sort</code>	Reversed	4.48 $\pm$ 0.17	8.43 $\pm$ 0.22	-88.2% †
<code>desc_reverse</code>	Random	5297.50 $\pm$ 27.50	2966.86 $\pm$ 17.17	+44.0% †
<code>desc_reverse</code>	Turtles	6421.62 $\pm$ 30.47	3826.81 $\pm$ 19.40	+40.4% †
<code>desc_reverse</code>	Zigzag	5299.78 $\pm$ 31.93	13.42 $\pm$ 0.12	+99.8% †
<code>desc_reverse</code>	Almost Sorted	174.43 $\pm$ 3.28	126.71 $\pm$ 2.56	+27.4% †
<code>desc_reverse</code>	Duplicates	3569.32 $\pm$ 17.29	1414.40 $\pm$ 8.50	+60.4% †
<code>desc_reverse</code>	Reversed	11.03 $\pm$ 0.32	11.86 $\pm$ 0.21	-7.5% †

against a topology-specific C_{pre} of 30.36 μs , so the sort phase itself slightly *improved* and the net degradation is pure overhead arithmetic.

4.3. Total Sorting Time and Per-Algorithm Analysis

Total-time evaluation reveals a dichotomy: adaptive quadratic algorithms gain wherever absolute inversion reduction ΔI is

Table 6. New topologies: mean time per execution (in μs , mean $\pm$ 95% CI half-width) and preprocessing gain for 10,000-element vectors, all eight methods. Same significance marks as Table 4.

Algorithm	Type	Pure	With pre	Gain
Merge Sort	Sawtooth	138.85 $\pm$ 0.69	143.46 $\pm$ 0.70	-3.3%*
Merge Sort	OrganPipe	119.53 $\pm$ 0.66	125.29 $\pm$ 0.81	-4.8%†
Merge Sort	FewRuns	115.81 $\pm$ 0.54	121.84 $\pm$ 0.41	-5.2%†
Merge Sort	Real	279.28 $\pm$ 1.11	279.26 $\pm$ 1.20	+0.0%*
Quicksort	Sawtooth	19.74 $\pm$ 0.19	26.48 $\pm$ 0.20	-34.2%†
Quicksort	OrganPipe	295.87 $\pm$ 2.20	298.74 $\pm$ 2.06	-1.0%†
Quicksort	FewRuns	300.73 $\pm$ 2.62	306.88 $\pm$ 2.83	-2.0%
Quicksort	Real	31.78 $\pm$ 0.21	39.17 $\pm$ 0.26	-23.3%†
Insertion Sort	Sawtooth	4820.54 $\pm$ 21.14	2297.88 $\pm$ 13.80	+52.3%†
Insertion Sort	OrganPipe	5168.19 $\pm$ 25.43	5166.41 $\pm$ 29.37	+0.0%
Insertion Sort	FewRuns	710.33 $\pm$ 10.82	692.84 $\pm$ 4.64	+2.5%
Insertion Sort	Real	4691.35 $\pm$ 37.82	2710.84 $\pm$ 14.21	+42.2%†
Bubble Sort	Sawtooth	18316.55 $\pm$ 110.17	12120.68 $\pm$ 67.03	+33.8%†
Bubble Sort	OrganPipe	17928.72 $\pm$ 51.51	18059.14 $\pm$ 49.91	-0.7%
Bubble Sort	FewRuns	3977.62 $\pm$ 21.96	3965.26 $\pm$ 16.84	+0.3%*
Bubble Sort	Real	21779.73 $\pm$ 66.43	18826.62 $\pm$ 63.05	+13.6%†
Selection Sort	Sawtooth	20595.88 $\pm$ 66.65	20682.48 $\pm$ 82.46	-0.4%
Selection Sort	OrganPipe	20004.16 $\pm$ 71.15	19909.36 $\pm$ 65.78	+0.5%
Selection Sort	FewRuns	20419.03 $\pm$ 55.93	20175.78 $\pm$ 55.75	+1.2%
Selection Sort	Real	20810.97 $\pm$ 72.65	20823.25 $\pm$ 68.90	-0.1%
sort_unstable	Sawtooth	23.16 $\pm$ 0.22	28.98 $\pm$ 0.14	-25.1%†
sort_unstable	OrganPipe	71.66 $\pm$ 0.33	76.19 $\pm$ 0.46	-6.3%†
sort_unstable	FewRuns	61.31 $\pm$ 0.28	66.90 $\pm$ 0.35	-9.1%†
sort_unstable	Real	29.29 $\pm$ 0.21	37.13 $\pm$ 0.26	-26.8%†
sort	Sawtooth	28.87 $\pm$ 0.40	32.55 $\pm$ 0.21	-12.8%†
sort	OrganPipe	20.00 $\pm$ 0.13	34.25 $\pm$ 0.14	-71.3%†
sort	FewRuns	29.62 $\pm$ 0.21	38.06 $\pm$ 0.19	-28.5%†
sort	Real	34.01 $\pm$ 0.41	39.27 $\pm$ 0.29	-15.5%†
desc_reverse	Sawtooth	4858.18 $\pm$ 36.03	2346.63 $\pm$ 33.87	+51.7%†
desc_reverse	OrganPipe	5172.80 $\pm$ 33.30	5221.97 $\pm$ 31.13	-0.9%
desc_reverse	FewRuns	695.96 $\pm$ 4.87	698.58 $\pm$ 5.46	-0.4%
desc_reverse	Real	4690.61 $\pm$ 28.75	2720.19 $\pm$ 21.05	+42.0%†

Table 7. Mean symmetric-preprocessing cost vs. vector size (mean over ten types; CLI tool, seed 42). Point $n=20,000$ included exclusively for linearity characterization.

Size	Pre (μs)	ns/element
1.000	1,13	1,13
5.000	5,50	1,10
10.000	11,51	1,15
20.000	23,09	1,15
100.000	113,58	1,14
1.000.000	946,36	0,95

large (14.9–100.0% at $n=10,000$), because the modest linear scan (≈ 1 ns/element) is amortized by suppressed local work. Mechanistically, *Insertion Sort* time per inversion is strikingly regular— ≈ 0.21 ns/inversion on Random, Turtles, Zigzag, Reversed, and Duplicates at $n=10,000$ (e.g. $5,209.69 \mu\text{s}/24,960,120$)—so its gains track ΔI directly (Almost Sorted deviates at 0.29 ns/inversion, consistent with overhead dominance at small I). *Bubble Sort* time, by contrast, is nearly flat (16–23 ms) across inputs whose I spans two orders of magnitude: its pass count follows the maximum left-displacement, not

Table 8. Isolated preprocessing cost C_{pre} per topology (same Criterion harness as the timing matrix; mean, μs). The CLI mean of Table 7 hides strong heterogeneity: unstructured inputs (Random, Duplicates) cost $\approx 5\times$ more than structured ones at $n=10,000$. Real has no 10^6 point (native asset cap).

Type	C_{pre} at 10^4	C_{pre} at 10^6
Random	30.36	3049.14
Turtles	5.66	537.28
Zigzag	5.72	545.30
Almost Sorted	7.31	713.73
Duplicates	27.63	2668.85
Reversed	5.69	517.05
Sawtooth	5.10	490.93
OrganPipe	5.86	589.98
FewRuns	5.49	558.90
Real	6.97	—

I . Hence its modest Random gain (+14.9% despite -45% inversions) versus collapse to ≈ 1 pass on aligned inputs ($\times 1,155$ – $2,280$ speedups). The new topologies revalidate the displacement mechanism: Sawtooth collapses like the aligned inputs (Insertion +52.3%, Bubble +33.8%), while OrganPipe and FewRuns—whose disorder the scan never engages ($\approx 0\%$ inversion reduction, maximum displacement unchanged)—stay

neutral for both (-0.7% to $+2.5\%$). In contrast, $O(n \log n)$ methods respond qualitatively distinctly.

Merge Sort shows mixed results under the paired criterion. Gain concentrates in topologies where preprocessing eliminates structural disorder: *Zigzag* ($+15.7\%$) and *Reversed* ($+3.3\%$) at $n = 10,000$, growing to $+18.6\%$ and $+6.0\%$ at $n=10^6$ (Table 9). In *Random*, *Turtles*, and *Almost Sorted*, however, the total degrades significantly. These degradations are arithmetic, as the per-topology decomposition of Table 8 shows rather than a disturbed key distribution.

Quicksort, implemented with three-way partitioning and median-of-three, shows systematic degradation under preprocessing. Loss is mild in most topologies (-4.5% to -8.0% at $n=10,000$ in *Random*, *Turtles*, and *Almost Sorted*; *Zigzag* is $+0.5\%$), but sharp in *Duplicates* (-34.4%) and *Reversed* (-38.4%). Again the overhead scale matters: Quicksort/Duplicates slows by $+14.25 \mu\text{s}$ at $n=10,000$ (topology-specific $C_{\text{pre}} 27.63 \mu\text{s}$) and $+1.24 \text{ ms}$ at $n=10^6$ (topology-specific $C_{\text{pre}} 2.67 \text{ ms}$), so most of the degradation is the preprocessing cost itself rather than a disturbed key distribution. The *Reversed* case additionally exposes an implementation-specific anomaly: the preprocessed (sorted) vector sorts *slower* (368.64 ms at $n=10^6$) than the random one (54.67 ms), and scaling from $n=10^4$ to 10^6 is $\approx 811\times$ on *Reversed* ($\approx n^{1.45}$) versus $\approx 145\times$ on *Random* ($\approx n^{1.08}$). Instrumented operation counts (faithful instrumented copies; the benchmarked algorithm untouched) locate the cause in degenerate partitioning—not recursion depth or branches. Comparisons per $n \ln n$ stay flat at 2.5 on *Random* ($n=10^4$ – 10^6) but climb $11.4 \rightarrow 27.6 \rightarrow 70.6$ on *Reversed*, $15.7 \rightarrow 38.4 \rightarrow 99.2$ on sorted input, and $12.7 \rightarrow 30.1 \rightarrow 77.3$ on *Zigzag*, while maximum depth stays shallow (7–9: the large side is iterated by tail-call, so no recursion pathology). The anatomy is telling: the root split of reversed input is balanced ($(100000, 50000, 49999)$ at $n=10^5$), but the recursion then follows a spine— $(49999, 49997, 1)$, $(24998, 1, 24996)$, $(24996, 2, 24993) \dots$ —median-of-three systematically electing near-extreme pivots inside the subintervals produced by the three-way partition. Against *sort_unstable* on the same pools, the implementation costs $\approx 5\times$ on *Random* but $30\times/141\times/465\times$ on *Reversed* and $43\times/209\times/749\times$ on sorted input ($n=10^4/10^5/10^6$). Since median-of-three favors sorted inputs in theory, this predicts the opposite of what is measured: the degeneracy is specific to this pivot/DNF interaction on runs, not a general property—so the -31 – -38% degradations are reported as implementation scope, with the exact DNF trigger left as future work.¹

Thus for the $O(n \log n)$ class, the technique is not beneficial in the evaluated implementation, with point exceptions (*Merge/Zigzag* and *Merge/Reversed* at both scales). Quicksort/AlmostSorted, the one apparent large-scale exception ($+5.5\%$ at $n=10^6$ after -4.5% at $n=10,000$ and -9.3% at $n=10^5$), flips sign across pools under the paired analysis and is therefore reported as non-robust rather than as a gain. These results delimit preprocessing applicability, indicating inversion reduction alone does not guarantee gains for this class.

Selection Sort behavior is consistent with its non-adaptive nature: since comparison count is fixed $(n(n-1)/2)$ regardless

of initial configuration, inversion reduction yields no systematic gain. Variations span -1.0% to $+3.1\%$; under the paired criterion *Random*, *Almost Sorted*, *Duplicates*, and *Zigzag* are non-significant, *Turtles* ($+3.1\%$) is significant in seed 42 but unstable across pools, and *Reversed* shows consistent degradation at $n=10,000$ ($19,150.38 \pm 353.49 \mu\text{s}$ vs. $20,900.45 \pm 113.30 \mu\text{s}$), reproduced at $n = 100,000$ (-12.8% , $1,849.65 \pm 8.34 \text{ ms}$ vs. $2,086.10 \pm 8.82 \text{ ms}$). Note the framing: the pure *Reversed* cell (19.15 ms) is the fast outlier among otherwise 20.2 – 21.5 ms cells, so the “degradation” is relative to an anomalously fast baseline. On the new topologies (*Sawtooth*, *OrganPipe*, *FewRuns*, *Real*) all variations are within -0.4% to $+1.2\%$ and non-significant—a clean neutrality confirmation. The *Reversed* mechanism is detailed below.

Since preprocessing transforms reversed input into an ordered sequence, minimum search per scan would find the smallest element at the first candidate; expected gain would be merely avoiding one swap per iteration, not justifying observed degradation. Instrumentation decides the mechanism (diagnostic harness with operation counters, same pools): the effect is real (≈ 9 – 13% , replicated round-robin) and rooted in inner-loop branch layout, not in misprediction rates. Operation counts at $n=10,000$ are exact: 50M comparisons in all three inputs, but minimum-updates of 78k (*random*) vs. 25M (*descending*) vs. 0 (*sorted*) and swaps of 9,989 vs. 5,000 vs. 0—the descending input sorts in $n/2$ swaps, each fixing two positions, with $(n/2)^2$ updates. Assembly inspection (`-C opt-level=3`, standalone probe, loop shape robust) shows the inner comparison compiles to a real conditional branch (`j1`) with asymmetric path costs (taken jumps to the loop top; not-taken executes a move plus a jump), giving a genuine input-dependent per-iteration cost (≈ 0.36 vs. 0.40 ns). Both branch patterns are stable, so a “more mispredictions” story is imprecise as stated; the refined taken-cheaper-than-not-taken layout effect and its quantification with hardware counters await Linux measurement. Either way the magnitude is bounded ($\approx 13\%$ max) with no systematic gain—consistent with the prediction that a non-adaptive algorithm does not exploit inversion reduction [5].

By simultaneously accessing opposite vector ends, symmetric preprocessing introduces additional control dependencies and increases instructions per element. On random instances, internal swap conditions (`if A[i] > A[j]`) become hard for the branch predictor [10, 13]. Consequently, where inversion reduction is modest, architectural penalties and extra instruction cost outweigh logical-restructuring benefits.

Fig. 3 synthesizes this dynamic: expressive exploitation in adaptive quadratic methods, mixed *Merge Sort* response, systematic three-way-*Quicksort* degradation, and neutral non-adaptive *Selection Sort* (except *Reversed*). Table 9 complements analysis for $n=10^6$ in quasilinear methods.

Values at $n=10^6$ replicate the $n=10,000$ pattern: *Merge Sort* gains restricted to *Zigzag* ($+18.6\%$), *Reversed* ($+6.0\%$), and *Duplicates* ($+4.8\%$, significant in seed 42 but unstable across pools), systematic *Quicksort* degradation (up to -37.7%), with *Turtles* ($+0.0\%$) and *Almost Sorted* ($+5.5\%$) failing the paired-robustness check, confirming applicability delimitation to adaptive methods. Under the paired criterion the non-significant cells at $n=10^6$ are *Merge/Random* (-0.9%) and *Quicksort/AlmostSorted*.

Fig. 4 extends the picture across collected sizes ($n=16$ – 10^7 , same Criterion means): *Insertion/Random* rises from $\approx 0\%$ at $n=16$ to a stable $\approx 44\%$ plateau from $n=10^4$; *Bubble/Random* is non-monotonic at small n (peak $+31.9\%$ at $n=64$) with an uptick at 10^5 (8.9 – 25.6%); *Merge/Zigzag* grows monotonically

¹ Times and inversion counts are direct measurements (Tables 3 and 4); the F2 diagnostic counts above come from the instrumented harness on identical pools. Branch-prediction attributions remain hypotheses, not direct measurements (unavailable on Windows, Section 3.5).

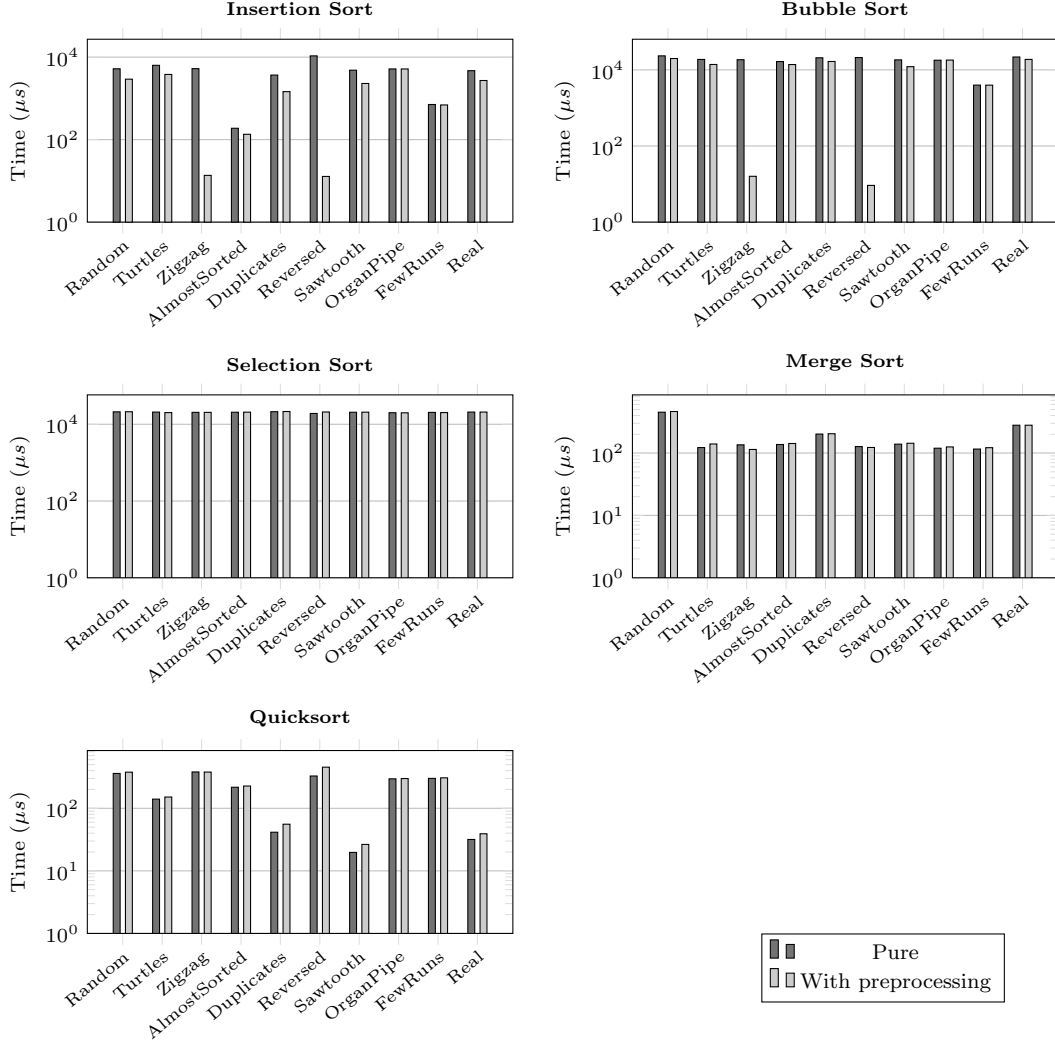


Figure 3. Temporal comparison between pure and symmetric-preprocessed sorting ($n = 10,000$) across all ten topologies, log scale.

(1.8–18.6%) while Merge/Reversed is negative at small n (down to -5.3%), positive mid-range ($+3.3$ – $+11.5\%$), and fades to -2.2% at 10^7 ; Quicksort/Random attenuates with size (-10.6 to -1.8%); Quicksort/AlmostSorted is non-monotonic and positive only at 10^6 ($+5.5\%$, non-robust across pools). Against the `sort_unstable` reference the method is deeply negative at every size (StdUnstable/Random from -0.9% at $n=16$ down to -77.5% at $n=1,000$), and Insertion/Sawtooth turns positive only above $n=128$ ($+21$ – $+53\%$) while Insertion/OrganPipe stays flat near zero—evidence that size trends do not uniformly stabilize and must be reported per topology.

5. Conclusion

This work presented and evaluated an $O(n)$ symmetric preprocessing for inversion-count reduction in sorting-algorithm input sequences. On most non-aligned inputs it removed 40–62% of inversions at ≈ 1.0 ns/element, yielding 14–60% gains in adaptive quadratic algorithms (Insertion Sort and Bubble Sort) at $n=10,000$; near-total inversion elimination and $\times 386$ – $2,280$ speedups occurred only on symmetry-aligned inputs (Reversed, Zigzag) and must not be generalized. Gains materialize when

sorting-cost reduction exceeds preliminary-step cost, satisfying $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$; for adaptive methods this takes the actionable form $\Delta I \gtrsim 5n$ (Section 3.4), obeyed by every $n=10,000$ outcome. Same-algorithm comparisons are reported alongside the fastest available sorters (`sort_unstable/sort`) and run-detection, which strictly delimit the practical scope (Section 4.2): the method never beats the standard sorter, and run-detection alone captures the aligned-input extreme.

For the evaluated non-adaptive quadratic algorithm, Selection Sort showed no systematic gain (-1.0% to $+3.1\%$, robustly significant only in Reversed under the paired rule; Turtles is significant in seed 42 but pool-unstable, Zigzag non-significant), except consistent ≈ 9 – 13% degradation on fully reversed input (paired 95% CIs: -9.1% at $n=10,000$, -12.8% at $n=100,000$), traced by operation counts and assembly inspection to inner-loop branch-layout asymmetry rather than misprediction rates (Section 4.3). This theoretically predicted neutrality via comparison-count invariance ($n(n-1)/2$) delimits that inversion reduction does not convert into gain for non-adaptive methods.

In evaluated $O(n \log n)$ algorithms, the method revealed important limitations. With the used Quicksort implementation

Table 9. Mean time (ms) and preprocessing gain for 1,000,000-element vectors—quasilinear methods and standard baselines (quadratic omitted for temporal infeasibility). Source: Criterion, 50 samples, 95% CI. Same paired-significance marks as Table 4; rows below the rule are the new topologies (Real excluded by the native asset cap).

Algorithm	Type	Pure	With pre	Gain
Merge Sort	Random	68.44 ± 0.80	69.04 ± 0.59	-0.9%
Merge Sort	Turtles	16.66 ± 0.16	18.57 ± 0.38	-11.5% [†]
Merge Sort	Zigzag	18.62 ± 0.30	15.15 ± 0.14	+18.6% [†]
Merge Sort	Almost Sorted	20.15 ± 0.30	20.57 ± 0.28	-2.1%*
Merge Sort	Duplicates	26.57 ± 0.33	25.29 ± 0.27	+4.8%*
Merge Sort	Reversed	15.98 ± 0.32	15.02 ± 0.13	+6.0% [†]
Quicksort	Random	53.27 ± 0.14	54.67 ± 0.11	-2.6% [†]
Quicksort	Turtles	101.17 ± 0.75	101.17 ± 0.77	+0.0%*
Quicksort	Zigzag	314.61 ± 2.36	328.79 ± 2.27	-4.5% [†]
Quicksort	Almost Sorted	32.74 ± 0.30	30.94 ± 0.26	+5.5%
Quicksort	Duplicates	3.97 ± 0.02	5.21 ± 0.02	-31.4% [†]
Quicksort	Reversed	267.65 ± 1.56	368.64 ± 1.04	-37.7% [†]
<code>sort.unstable</code>	Random	10.81 ± 0.07	13.80 ± 0.06	-27.6% [†]
<code>sort.unstable</code>	Turtles	6.77 ± 0.02	7.02 ± 0.03	-3.6%*
<code>sort.unstable</code>	Zigzag	12.07 ± 0.14	9.66 ± 0.04	+20.0% [†]
<code>sort.unstable</code>	Almost Sorted	10.99 ± 0.03	11.51 ± 0.03	-4.8% [†]
<code>sort.unstable</code>	Duplicates	1.39 ± 0.01	4.10 ± 0.02	-194.3% [†]
<code>sort.unstable</code>	Reversed	0.40 ± 0.01	0.76 ± 0.01	-87.1% [†]
<code>sort</code>	Random	15.10 ± 0.04	18.02 ± 0.10	-19.4% [†]
<code>sort</code>	Turtles	3.02 ± 0.02	3.12 ± 0.02	-3.4%*
<code>sort</code>	Zigzag	15.54 ± 0.05	6.60 ± 0.03	+57.5% [†]
<code>sort</code>	Almost Sorted	11.60 ± 0.16	12.06 ± 0.04	-4.0% [†]
<code>sort</code>	Duplicates	2.53 ± 0.02	5.01 ± 0.03	-98.0% [†]
<code>sort</code>	Reversed	0.42 ± 0.00	0.79 ± 0.01	-88.6% [†]
Merge Sort	Sawtooth	18.86 ± 0.15	19.29 ± 0.20	-2.3%
Merge Sort	OrganPipe	15.56 ± 0.20	16.01 ± 0.15	-2.9%
Merge Sort	FewRuns	14.90 ± 0.26	15.56 ± 0.21	-4.4%
Quicksort	Sawtooth	2.94 ± 0.01	3.22 ± 0.01	-9.3% [†]
Quicksort	OrganPipe	4070.08 ± 9.70	4071.58 ± 10.51	-0.0%*
Quicksort	FewRuns	248.60 ± 0.97	248.97 ± 0.94	-0.1%
<code>sort.unstable</code>	Sawtooth	2.52 ± 0.01	2.77 ± 0.01	-10.2%*
<code>sort.unstable</code>	OrganPipe	11.98 ± 0.03	12.53 ± 0.03	-4.6%*
<code>sort.unstable</code>	FewRuns	9.35 ± 0.02	9.82 ± 0.02	-5.0% [†]
<code>sort</code>	Sawtooth	3.53 ± 0.01	3.53 ± 0.01	+0.0%
<code>sort</code>	OrganPipe	2.45 ± 0.06	3.88 ± 0.05	-58.1% [†]
<code>sort</code>	FewRuns	3.25 ± 0.01	4.00 ± 0.01	-22.9% [†]

(three-way partitioning and median-of-three), preprocessing caused systematic degradation—about 4%–8% at $n=10,000$ in most topologies (-4.5% for Zigzag at $n=10^6$) and $\approx 31\%$ – 38% in *Duplicates* and *Reversed*—largely on the scale of preprocessing overhead itself, plus an ordered-input degeneracy diagnosed as pivot/DNF interaction (comparisons per $n \ln n$ climbing $11 \rightarrow 71$ on Reversed while depth stays shallow), whose exact trigger remains future work.

In Merge Sort effects were mixed, with gains restricted to topologies most structurally affected by symmetric scanning (Zigzag $+15.7\%$ at $n=10,000$ and $+18.6\%$ at $n=10^6$; Reversed $+3.3\%$ and $+6.0\%$) and degradation elsewhere. Practical viability must therefore be strictly conditioned on adaptive quadratic algorithms, not extending to quasilinear methods generally and particularly not benefiting non-adaptive algorithms like Selection Sort.

As future work, we recommend (i) closed-form theoretical derivation of expected inversion reduction under random input in Hwang et al.’s framework [7], including the

swap-monotonicity lemma sketched in Section 3.4; (ii) a formal gap analysis against Shell-type passes and validation of the Quicksort anomaly against a reference implementation (pdqsort); (iii) second-hardware and Linux-environment tests with `perf_event` infrastructure, enabling direct *cache-miss* and *branch-misprediction* measurement; and (iv) alternative partitionings and adaptive techniques such as *BlockQuicksort* [10] and *Timsort* [11] as comparative baselines.

References

1. D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Reading, MA: Addison-Wesley, 1973.
2. R. Sedgewick and K. D. Wayne, *Algorithms*, 4th ed. Upper Saddle River: Addison-Wesley, 2011.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
4. N. Ziviani, *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning, Aug. 2021.

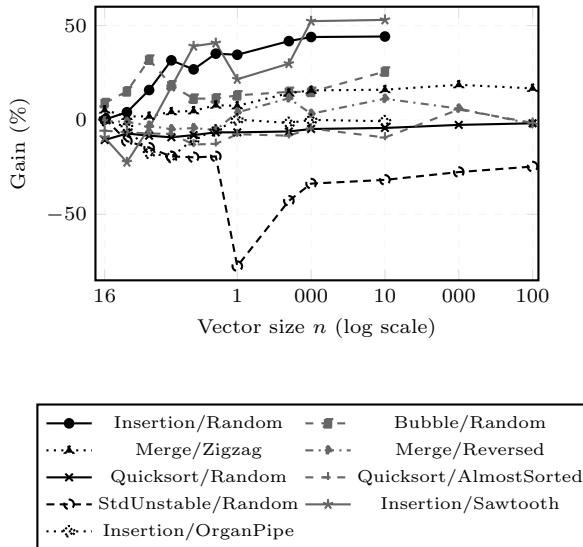


Figure 4. Preprocessing gain vs. input size (Criterion means; quadratic methods capped at $n=10^5$; gaps are unplanned cells).

5. V. Estivill-Castro and D. Wood, “A survey of adaptive sorting algorithms,” *ACM Computing Surveys*, vol. 24, no. 4, pp. 441–476, Dec. 1992, doi: <https://doi.org/10.1145/146370.146381>. [Online]. Available: <https://dl.acm.org/doi/10.1145/146370.146381>
6. H. Mannila, “Measures of presortedness and optimal sorting algorithms,” *IEEE Transactions on Computers*, vol. C-34, no. 4, pp. 318–325, 1985, preliminary version in Proc. 11th ICALP, LNCS 172, pp. 324–336, 1984. doi: <https://doi.org/10.1109/TC.1985.5009382>.
7. H.-K. Hwang, B.-Y. Yang, and Y.-N. Yeh, “Presorting algorithms: An average-case point of view,” *Theoretical Computer Science*, vol. 242, no. 1-2, pp. 29–40, Jul. 2000, doi: [https://doi.org/10.1016/S0304-3975\(98\)00181-9](https://doi.org/10.1016/S0304-3975(98)00181-9). [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0304397598001819>
8. A. C. Gil, *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda, Feb. 2019.
9. R. H. Sampieri, C. F. Collado, and M. d. P. B. Lucio, *Metodologia de pesquisa*. Penso, Nov. 2021.
10. S. Edelkamp and A. Weiß, “Blockquicksort: Avoiding branch mispredictions in quicksort,” in *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, ser. LIPIcs, vol. 57, 2016, pp. 38:1–38:16, doi: <https://doi.org/10.4230/LIPIcs.ESA.2016.38>. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2016.38>
11. N. Auger, V. Jugé, C. Nicaud, and C. Pivoteau, “On the worst-case complexity of timsort,” in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, ser. LIPIcs, vol. 112, 2018, pp. 4:1–4:13, doi: <https://doi.org/10.4230/LIPIcs.ESA.2018.4>. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2018.4>
12. A. LaMarca and R. E. Ladner, “The influence of caches on the performance of sorting,” *Journal of Algorithms*, vol. 31, no. 1, pp. 66–104, Apr. 1999, doi: <https://doi.org/10.1006/jagm.1998.0985>. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0196677498909853>
13. K. Kaligosi and P. Sanders, “How branch mispredictions affect quicksort,” in *Proceedings of the 14th Annual European Symposium on Algorithms (ESA 2006)*, ser. LNCS, vol. 4168. Springer, 2006, pp. 780–791, doi: https://doi.org/10.1007/11841036_69.
14. “Presorting algorithm – experimental framework and data,” Anonymous repository for double-blind review, 2026, source code and experimental data available upon acceptance.